

```

7. #include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

// Global variable
int global_var = 100;

// Static variable
static int static_var = 50;

int main()
{
    // Stack variable
    int stack_var = 10;

    // Heap variable
    int *heap_var = (int *)malloc(sizeof(int));

    if (heap_var == NULL)
    {
        printf("Memory allocation failed!\n");
        return 1;
    }

    *heap_var = 200;

    printf("\n===== \n");
    printf("    LINUX PROCESS MEMORY LAYOUT\n");
    printf("===== \n");

    printf("Process ID (PID)    : %d\n", getpid());
    printf("Code Address       : %p\n", (void *)main);
    printf("Global Address      : %p\n", (void *)&global_var);
    printf("Static Address      : %p\n", (void *)&static_var);
    printf("Heap Address        : %p\n", (void *)heap_var);
    printf("Stack Address       : %p\n", (void *)&stack_var);

    printf("\nVariable Values:\n");
    printf("Global = %d\n", global_var);
    printf("Static = %d\n", static_var);
    printf("Heap   = %d\n", *heap_var);
    printf("Stack  = %d\n", stack_var);

    printf("\n===== \n");
    printf("Process is running...\n");
    printf("Use this command in another terminal:\n");
    printf("cat /proc/%d/maps\n", getpid());
    printf("===== \n");

    printf("\nPress ENTER to terminate the process...");
    getchar();
}

```

```

free(heap_var);

return 0;
}

```

```

saranya@saisaranya:~$ nano heap.c
saranya@saisaranya:~$ gcc heap.c -o shell
saranya@saisaranya:~$ ./shell

=====
      LINUX PROCESS MEMORY LAYOUT
=====
Process ID (PID)       : 486
Code Address           : 0x5d073eac3209
Global Address         : 0x5d073eac6010
Static Address         : 0x5d073eac6014
Heap Address           : 0x5d0762850010
Stack Address          : 0x7ffea40b513c

Variable Values:
Global = 100
Static = 50
Heap   = 200
Stack  = 10

=====
Process is running...
Use this command in another terminal:
cat /proc/486/maps
=====

Press ENTER to terminate the process...|

```

8a. #include <stdio.h>

#include <stdlib.h>

int main()

{

int *a, *b;

// malloc: allocate memory for 5 integers

a = (int *)malloc(5 * sizeof(int));

if (a == NULL)

{

```
    printf("malloc failed\n");  
    return 1;  
}
```

```
printf("malloc: allocated memory for 5 integers\n");
```

```
// calloc: allocate and initialize 5 integers to zero
```

```
b = (int *)calloc(5, sizeof(int));
```

```
if (b == NULL)  
{  
    printf("calloc failed\n");  
    free(a);  
    return 1;  
}
```

```
printf("calloc: allocated memory for 5 integers\n");
```

```
// realloc: increase a from 5 to 10 integers
```

```
a = (int *)realloc(a, 10 * sizeof(int));
```

```
if (a == NULL)  
{  
    printf("realloc failed\n");  
    free(b);  
    return 1;  
}
```

```
printf("realloc: increased memory from 5 to 10 integers\n");
```

```
// free: release allocated memory
```

```
free(a);
```

```

printf("free: released malloc/realloc memory\n");

free(b);

printf("free: released calloc memory\n");

printf("\nMemory allocation completed successfully.\n");

return 0;
}

```

```

saranya@saisaranya:~$ nano memory_alloc.c
saranya@saisaranya:~$ gcc memory_alloc.c -o memory_alloc
saranya@saisaranya:~$ ./memory_alloc
malloc: allocated memory for 5 integers
calloc: allocated memory for 5 integers
realloc: increased memory from 5 to 10 integers
free: released malloc/realloc memory
free: released calloc memory

Memory allocation completed successfully.
saranya@saisaranya:~$ |

```

```

8b#include <stdio.h>

```

```

#include <unistd.h>

```

```

#include <sys/wait.h>

```

```

int main()

```

```

{

```

```

    int x = 100;

```

```

    printf("Before fork: x = %d\n", x);

```

```

    pid_t pid = fork();

```

```

    if (pid < 0)

```

```

    {

```

```

        printf("Fork failed!\n");

```

```
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Before Modification: x = %d\n", x);

        x = 500;

        printf("Child After Modification: x = %d\n", x);
        printf("Child PID: %d\n", getpid());
    }
    else
    {
        sleep(2);

        printf("Parent After Child Modification: x = %d\n", x);
        printf("Parent PID: %d\n", getpid());

        wait(NULL);
    }

    return 0;
}
```

```
saranya@saisaranya:~$ nano cow.c
saranya@saisaranya:~$ gcc cow.c -o cow
saranya@saisaranya:~$ ./cow
Before fork: x = 100
Child Before Modification: x = 100
Child After Modification: x = 500
Child PID: 487
Parent After Child Modification: x = 100
Parent PID: 486
saranya@saisaranya:~$ |
```